

2026 国庆 CSP-S 模拟赛 Day1 复盘

比赛：代码源新 OJ · 2026 国庆 CSP-S 模拟赛 day1 · 4 题 · 4 小时（北京时间 19:04 到 23:04）结果：45 / 400（A 20 TLE，B 25 WA，C 0 未交，D 0 未交）分析依据：归档（题面、2 份提交源码）、赛时录音转写、虚拟机赛前/赛后桌面导出（含回收站、下载的大样例、checker）、4 张草稿纸。四题正解我都写了，并用你下载的大样例和暴力对拍验证过。

零、一页结论

这场不是不会，是三个具体错误叠在一起：

1. 读题漏了 $\sum n \leq 8000$ （录音 18:24），把目标定成 $O(n)$ 。T1 本来 $O(n^2)$ 就能过，你为一个不存在的要求花了近 2 小时。
2. 你 T1 的 $O(n^2)$ 思路其实是对的，死在一个 `distance` 上。`std::distance` 在 `set/multiset` 上是一步一步数的，每次 $O(n)$ ，整体变成 $O(n^3)$ 。我把你赛时交的那份代码只把 `multiset` 换成树状数组，别的一行不动，最大数据 0.57 秒（时限 1.5 秒），就是 100 分。这一处丢了 80 分。
3. C 和 D 基本没碰，但它们是这场最该拿分的题。
 - C 是一个 $O(n)$ 的树形 DP。子任务 1 的答案是 $\sum m$ ，子任务 2 的答案是 $\sum \max(m, w)$ ，各一行代码各 10 分。
 - D 你在录音 3:06 到 3:12 已经把模型推对了（拆成环、换成质因子、想到小凯的疑惑）。直接写个完全背包就是 45 分，最后一行代码都没交。

按你现有的知识，这场的合理分数：

题	实际	能力范围内能拿	差在哪
A	20	100	读漏 $\sum n + \text{distance}$
B	25	40	checker 里写着 YES/NO 条件没看出来； $\sum n \leq 8$ 能暴搜
C	0	70	只看了 6 分钟；两个一行送分 + 每次询问重做一遍 DP
D	0	45	模型对了没写背包
合计	45	约 255	

一、证据完整性检查

已看到：

- 归档：4 题题面、时空限制、子任务表；2 次提交的源码和分数
- 录音转写 2 份。它们是同一段录音的两个转写版本，第一份（23:08 导出的“赛时音频_原文”）质量好得多，下文时间都以它为准

- 虚拟机导出：赛前桌面、赛后桌面、回收站（T1 的两个旧版本）、Downloads（四题大样例 + B 题 checker.cpp）
- 草稿纸 4 张

没看到：

- 榜单和排名：归档时"排行榜"没勾选，所以没有名次，也没有同分段对比
- 其他选手的代码
- 录音 3:21:35 到 3:31:59 有 10 分钟空白（在写 B 的骗分）

我纠正的数据：

项目	归档里写的	实际	怎么查出来的
提交时间	"时间未知"	A：21:17:19；B：22:38:18	Hydro 提交 ID 的前 8 位是时间戳（6ab6745f、6ab6875a）
时区	America/New_York	你在北京时间	虚拟机时区设错了，所有文件时间比北京慢 12 小时。开赛北京 19:04，文件时间显示 07:01
录音起点	—	北京 18:52:24 左右	两处对齐：录音 09:24 说"脚本已完成"，和模板文件时间 19:01:48 吻合；录音 1:28:11 下载 T1 附件，和 zip 文件时间 20:20:44 吻合

下文时间写法：T+1:23 = 开赛后 1 小时 23 分（开赛 = 录音 11:56 = 北京 19:04）。

二、比赛总画像

题	名字	时限 / 内存	本质	你的状态	分
A	蜗蜗的记录带	1.5s / 1024MiB	翻转区间后逆序对最大； $\sum n \leq 8000, O(n^2)$	1 次提交，TLE	20
B	蜗蜗的联动面板	1s / 256MiB	构造：判定条件 $a_i \leq i$ ，然后归纳构造	1 次提交，骗分	25
C	蜗蜗的晶体树阵	1s / 512MiB	树形 DP（支配型三状态）+ 按深度分层	看了约 10 分钟，未交	0
D	蜗蜗的轮换密钥	5s / 256MiB	环长整除 $k \rightarrow$ 质因子凑 $n \rightarrow$ 同余最短路	模型推对，未交	0

一句话判断：建模能力其实在线（A 的区间内顺逆互换、D 的环分解都是自己推出来的），但读题、复杂度常识、时间分配、止损四个环节全部失守，外加 IDE 故障吃掉约半小时。

时间都花在哪了：

去向	时长	占比
T1（含 IDE 故障约 30 分钟）	2 小时 13 分	55%
T2（读题找性质 23 分钟 + 骗分 30 分钟 + 其他）	约 57 分钟	24%
T4	约 25 分钟	10%
T3	约 10 分钟	4%
最后放弃等结束	约 8 分钟	3%

三、时间线

开赛后	北京时间	发生了什么	评价
赛前	18:57 至 19:02	建 4 个对拍文件夹 + 1 个 debug 工程，用了约 5 分钟	四个对拍文件夹里的 ac/wa/gen 从头到尾是空模板，一次没用
T+0:00	19:04	开赛读 T1。说"先一次性把所有题目全部打开"，但只读了 T1	✗ 没通读
T+0:03 至 0:06		推出"区间内顺序对和逆序对互换，区间外不变"	✓ 关键结论正确
T+0:06	19:10	算复杂度："10 的 5 次方组数据乘以 8000……所以这一题我要做到 $O(n)$ "	✗ 全场最贵的一句话：漏看 $\sum n \leq 8000$
T+0:10 至 0:14		列出"净收益 = $x - y$ "，写前缀公式（草稿纸 1）	公式本身就不成立，见第四节 A
T+0:14 至 0:30		想"前面有几个比我大"：二分、链表、set，最后用 multiset + distance	没想起树状数组
T+0:30 至 0:53	19:34 至 19:57	Code::Blocks 故障：没反应、弹两个窗口、"这代码不是我的"、每改一次要重启 IDE	约 23 分钟
T+0:53 至 1:15		调公式正负号，LST/RST 来回互换	草稿纸和代码里 lst 的含义是反的，见第四节 A
T+0:58	20:02	样例输出 20，但 $n=5$ 最多只有 10 对："第二个不是最多十个吗？怎么出现 20 了"	止损信号 1：答案超过理论上限，说明公式结构错了，不是符号错
T+1:16	20:20	下载大样例，第 1 组就错	止损信号 2
T+1:33	20:37	把 MA 初值从 0 改成 $-\text{INT_MAX}$	止损信号 3：开始靠改初值碰运气
T+1:44 至 1:50		大样例只有第 2 组（全递增）对	全递增时公式碰巧正确
T+1:54	20:58	看到 $\sum n \leq 8000$ ："完了全完了"	发现得太晚
T+1:56 至 2:05	21:00 至 21:10	写 $O(n^2)$ 的 multiset 版（写在 T2.cpp 里）	思路对，数据结构错
T+2:09	21:14	3-1 跑了 7.7 秒："八成是我机子内核少，跑的慢"	✗ 误判，见第五节
T+2:12	21:16	4-1 卡死："掩耳盗铃……I'm always right"	知道会超时还是交了
T+2:13	21:17:19	提交 A $\rightarrow$ 20 分 TLE	
T+2:18 至 2:41		T2：找性质、建 $i \rightarrow a_i$ 的图（草稿纸 3、4）、看 checker	checker 里写着判定条件，没看出来
T+2:41 至 2:47		T3：只看了 6 分钟，"八成是换根""像 DP 但难写"	✗ 判断错，是固定根

开赛后	北京时间	发生了什么	评价
T+2:48 至 3:04		T4：从样例找规律 → 拆成环 → 拆成质数 → 想到小凯的疑惑	✓ 模型基本对
T+3:04	22:08	"T4 我掰不动"，转去 T2 骗分	✗ 背包 45 分就在眼前
T+3:34	22:38:18	提交 B → 25 分 (特判 $a_i=i$ 和全 1)	
T+3:39 至 3:43		T3 看 $w=1$ 的样例，没写代码	
T+3:43 至 3:52		T4 写全排列暴力 (没写完，也编不过)	
T+3:52 至 4:00		"感觉可以等死了"	

四、逐题复盘

A | 蜗蜗的记录带 (20 → 本可以 100)

题目本质：翻转一个区间以后，逆序对最多是多少。 $\sum n \leq 8000$ ，所以 $O(n^2)$ 就够。

你想对的部分 (录音 16:26 至 17:49，草稿纸 1)：

- 翻转 $[l, r]$ 后，区间内原来的顺序对全变成逆序对，逆序对全变成顺序对
- 一个在区间内、一个在区间外的点对，以及两个都在区间外的点对，都不受影响
- 所以净收益 = 区间内顺序对数 - 区间内逆序对数，也就是你写的 $x - y$

这是整道题的核心结论，你 6 分钟就推出来了。

偏差 1：漏读 $\sum n \leq 8000$ (最贵)

录音 18:24 你按 "T 最多 10^5 ，n 最多 8000" 估了 8×10^8 ，于是认定必须 $O(n)$ 。数据范围最后一行写着 $\sum n \leq 8000$ 。一旦看到它， $O(n^2) = 6.4 \times 10^7$ ，完全够。

这一句把你带进了一条根本不存在的路，吃掉约 1 小时 50 分。

偏差 2：前缀逆序对相减 $\neq$ 区间逆序对

你的公式 (草稿纸 1)：

区间 $[i, j]$ 的收益 = $(rst[j] - rst[i-1]) - (lst[j] - lst[i-1])$ ，其中 lst/rst 是前缀逆序对数/前缀顺序对数

为什么它看起来对：前缀和相减求区间和，这个套路你用过很多次。

为什么不成立：前缀和相减只对 "每个元素自己贡献的量" 成立。逆序对是点对。

用 $P(r)$ 表示 $[1, r]$ 里的点对数，那么：

$$P(r) - P(l-1) = [l, r] \text{ 内部的点对} + \text{一端在 } [1, l-1]、\text{一端在 } [l, r] \text{ 的跨界点对}$$

跨界点对被算进去了，公式就错了。

最小反例：`[3, 1, 2]`

- 正确答案是 3：翻转 `[2, 3]` 得到 `[3, 2, 1]`
- 你的最终版输出 2
- 原因：区间 `[2, 3] = [1, 2]` 里有 1 个顺序对、0 个逆序对，真实收益 +1。但前缀相减会把位置 1 的 3 和 1、2 组成的两个跨界逆序对也算进来，得到 -1

我用全排列暴力测了你的最终版： $n=3$ 错 $1/6$ ， $n=4$ 错 $7/24$ ， $n=6$ 错 $378/720$ 。

`[2, 1, 3, 4]` 更离谱：你输出 6，可 $n=4$ 最多只有 6 对，而正确答案是 5。输出超过理论上限，就说明公式本身是错的，不是正负号的问题。

这正是你 T+0:58 遇到的"怎么出现 20 了"，当时就该停下来重新建模。

偏差 3：草稿纸和代码里 `lst` 的含义是反的

- 草稿纸 2 写的是："顺 `lst` 0 0 2 / 逆 `rst` 0 1 1"
- 代码里 `lst[i] = lst[i-1] + num`，`num` 是前面比我大的个数，所以 `lst` 是逆序

录音里你前后四次说"我搞反了"（1:14:04、1:20:06、1:21:04 "我把一开始那个 bug 又改了回来"、1:35:36），每次都在两套定义之间切换。

规则：变量含义写在代码第一行注释里，草稿纸和代码用同一个名字。

偏差 4：`std::distance` 在 `multiset` 上是 $O(n)$

你在 T+1:56 改写的 $O(n^2)$ 版本，思路完全正确：枚举左端点 i ，右端点 j 往右扩，边扩边维护区间内的逆序对。

问题出在 `distance (stl.upper_bound(a[j]), stl.end())`：

- `set/multiset` 的迭代器只能一步一步走，`distance` 是 $O(\text{距离})$ ，不是 $O(\log n)$
- 整体接近 $O(n^3)$
- 就算没有 `distance`，`multiset` 的插入常数也很大： $p_i = i$ 的数据里 `distance` 每次是 $O(i)$ ， $n=8000$ 照样跑 6.1 秒，所以子任务 2 也挂了，最终只拿到子任务 1 的 20 分

实测（和赛时同一份代码，在一台正常的 4 核服务器上跑）：

大样例	n	你交的代码	只换树状数组	$O(n^2)$ 正解
1	100	0.009s	0.006s	0.007s
2	多组全递增	0.34s	0.03s	0.02s

大样例	n	你交的代码	只换树状数组	$O(n^2)$ 正解
3	2000	9.2s	0.02s	0.01s
4	5000	>60s	0.20s	0.06s
5	8000	>60s	0.57s	0.11s

两份代码都在 [正解代码/](#) 里：

- [A_记录带_赛时代码只换树状数组.cpp](#)：你的原结构，只把 multiset 换成树状数组，约 15 行改动
- [A_记录带_0n2.cpp](#)：纯 $O(n^2)$

正解 ($O(n^2)$)，不需要任何数据结构)

设 $inv(l, r)$ 为区间 $[l, r]$ 的逆序对数。固定右端点 r ，让 l 从 $r-1$ 往左走，用 cnt 累计 $[l, r-1]$ 里比 $a[r]$ 大的个数：

```
inv(l, r) = inv(l, r-1) + cnt
```

用一个一维数组 $f[l]$ 存 $inv(l, \text{当前 } r)$ ，每次加 cnt 就行。收益 = $C(\text{len}, 2) - 2 \cdot inv(l, r)$ 。答案 = 原逆序对数 + $\max(0, \text{最大收益})$ 。

下次识别信号：

- 数据范围里出现 $\sum n \leq X$ → 复杂度按 X 算，不按 $T \times n$ 算
- "区间里的点对统计" (逆序对、相等对等) → 不能前缀相减，要么固定一端增量维护，要么二维 DP
- 要"前面有几个比我大" → 树状数组，不是 set

B | 蜗蜗的联动面板 (25 → 本可以 40, 满分可达)

题目本质：构造题。先判断有没有解，再构造操作序列。

你当时的状态：

- 找到两个正确的必要条件："A_n 必须 $\leq n$ "、"a_1 只能被值为 1 的人改"
- 把问题建成图 $i \rightarrow a_i$ (草稿纸 3、4)
- 读了 checker (录音 2:51 "这个 checker 看明白了")
- 最后特判 $a_i = i$ 和全 1，拿了 25 分
- 你的代码永远输出 YES，所以所有有 NO 的子任务都拿不到

被错过的最大信号：checker 里直接写了判定条件

你下载的 [checker.cpp](#) 第 17 到 22 行：

```
static bool feasible(const TestCase& tc) {
    for (int i = 1; i <= tc.n; ++i) {
        if (tc.a[i] > i) return false;
    }
    return true;
}
```

有解当且仅当对所有 i 都有 $a_i \leq i$ 。你当时在找"第一个 NO 为啥"，答案就写在这 6 行里。

以后拿到 checker，第一件事就是搜 **NO** / **feasible** / **impossible** 这几个词。

另外 checker 还校验了 $K == n(n+1)/2 - \sum a$ ，说明操作次数是固定的。题面里的 $K \leq n(n-1)/2$ 其实是自动满足的，不是需要你去控制的限制。

为什么是 $a_i \leq i$ (必要性)：数值只增不减。如果某个 $a_i > i$ ，由于数组有序，位置 i 到 n 这 $n-i+1$ 个数最后都 $\geq a_i \geq i+1$ ，但 $[i+1, n]$ 里只有 $n-i$ 个不同的值，塞不下。这是抽屉原理。

我用暴力搜索验证过： $n \leq 7$ 的全部 2353 个有序数组，这个条件和"是否有解"完全一致。

正解构造：归纳 + 链式上推

归纳假设：前 $i-1$ 个位置已经变成 1 到 $i-1$ 的一个排列。现在加入位置 i ，它的值是 $t = a_i \leq i$ 。

- 如果 $t = i$ ，直接成立
- 否则值 t 出现了两次（原来的持有者，加上位置 i ），值 i 还没有人。按 $v = t, t+1, \dots, i-1$ 的顺序，每次把"原来持有 v 的那个位置"加 1，推到 $v+1$

为什么每一步都合法：推的时候，1 到 $i-1$ 这些值始终都有人持有（被推走的那个值，由刚推上来的那个位置或位置 i 顶上）。要给位置 x 加 1，需要人的值等于 x ，而 $x \leq i-1$ ，一定有人持有。

总操作数正好是 $\sum(i - a_i) = K$ ，复杂度 $O(n^2)$ 。代码见 [正解代码/B_联动面板.cpp](#)。

验证：

- 5 组大样例全部通过（用 checker 的逻辑逐条模拟）
- $n \leq 8$ 、值不超过 $n+2$ 的全部 33097 个数组，全部通过

你能拿但没拿的分：

- 子任务 1 ($\sum n \leq 8$ ，15 分)：直接 DFS 搜索，状态最多 8^8 个，还能剪枝
- 你的 25 分再加 15 分，就是 40

下次识别信号：

- 构造题先解决判定：写暴力、打表、看 checker
- "所有数只增不减 + 最后变成排列" → 想抽屉原理
- 构造没思路 → 试试"前 $i-1$ 个已经排好，加入第 i 个怎么修"

C | 蜗蜗的晶体树阵 (0 → 本可以 70, 满分也不难)

题目本质：以 k 为根的树形 DP。"强共振需要一个核邻居"这一类，就是经典的"保安站岗 / 最小支配集"型三状态 DP。半径 L 只影响深度 $\leq L$ 的点。

你当时的状态：看了 6 分钟 (T+2:41 至 2:47)。判断"八成是换根""感觉像 DP 但难写"，然后去看 T4。T+3:40 回来翻大样例，发现"后面输出都是同一个数……估计就是达到上限了"。

这个观察是对的： L 超过最大深度以后，所有点都在半径内，答案固定为 $\Sigma \max(m, w)$ 。这正是子任务 2 的解法，你已经看出来了，只差把它写成代码。

偏掉的地方：

- 不是换根：根是固定的（调谐器接在 k ），直接以 k 为根就行
- 子任务 1、2 都是送分题，各 10 分，各一行代码：
 - 子任务 1 ($w_i = 1$)： $m_i \geq 1 = w_i$ ，共振永远不比恒稳亮，当核更是 0，答案恒为 Σm_i
 - 子任务 2 ($q=1$ 且 $L=n$)：所有点都在半径内，都能自由共振，答案是 $\Sigma \max(m_i, w_i)$
- 子任务 3、4 ($n, q \leq 5000$)：每次询问重做一遍 $O(n)$ 的 DP， $O(nq) = 2.5 \times 10^7$ ，拿 $10+40 = 50$ 分
- 加起来 70 分

正解 ($O(n + q)$)：以 k 为根，对每个点 v 求：

- $g0(v)$ ： v 的父亲不是核时， v 子树的最大亮度
- $g1(v)$ ： v 的父亲是核时（或者 v 在半径内，可以自由共振）， v 子树的最大亮度
- $core(v)$ ： v 自己当核， $= \Sigma g1(\text{儿子})$

转移：

- $g0(v) = \max(core(v), m_v + \Sigma g0(\text{儿子}), w_v + \Sigma g0(\text{儿子}) + \max \text{ over 儿子 } c \text{ of } (core(c) - g0(c)))$
 - 第三项要求至少一个儿子当核
- $g1(v) = \max(core(v), \max(m_v, w_v) + \Sigma g0(\text{儿子}))$

询问 L 的答案 = 深度 $< L$ 的所有点的 $\max(m, w)$ 之和 + 深度恰好为 L 的所有点的 $g1$ 之和。

理由：

- 深度 $< L$ 的点，自己和所有邻居都在半径内，当核没有任何好处（亮度 0，邻居本来就能共振），所以直接取 $\max(m, w)$
- 深度 $= L$ 的点能自由共振，它的状态等价于"父亲是核"，也就是 $g1$ 。但它也可能值得当核，去支撑深度 $L+1$ 的儿子，这种情况已经包含在 $g1$ 的 $core$ 选项里

预处理每一层的和，每次询问 $O(1)$ 。

验证：4 组大样例全部通过；300 棵随机小树 ($n \leq 7$) 和 3^n 暴力比对，全部一致。代码见 `正解代码/C_晶体树阵.cpp`。

下次识别信号：

- "某点要有一个邻居是 X 才能选 Y " → 支配型 DP，三状态（自己是 X / 被儿子支撑 / 等父亲支撑）

- "距离某点 $\leq L$ 的部分被豁免" $\rightarrow$ 以那个点为根，按深度分层
- 读到子任务表里有 " $L = n$ " 这种条件，先想它是不是让题目退化成了平凡情况

D | 蜗蜗的轮换密钥 (0 $\rightarrow$ 本可以 45，满分是经典套路)

题目本质： $f^k = \text{恒等}$ ，所以 f 是一个排列；没有不动点，所以每个环长 ≥ 2 ；每个环长都要整除 k 。问题变成： n 能不能写成若干个 " k 的约数 (≥ 2)" 之和。

任何 ≥ 2 的约数都是 k 的某个质因子的倍数，一个长 d 的环可以拆成 d/p 个长 p 的环。所以问题进一步等价于： n 能不能写成 k 的质因子的非负整数组合。这是 Codeforces 986F 的原题。

你推对的部分 (录音 3:06 至 3:12)：

- "能不能让你拆分成若干个环" $\checkmark$
- "能否拆成若干个质数" $\checkmark$
- "这跟小凯的那个钱有关" $\checkmark$ ：两个质因子时，就是 NOIP 2017 小凯的疑惑
- "第一步把 k 分解质因数" $\checkmark$

偏掉的地方：

- 草稿纸 4 写的是 " $P_1 + \dots + P_k = n$, $P_1 \dots P_k$ 互质, $P_1 \times P_2 \times \dots \times P_k = k$ "。乘积等于 k 是多余的限制。题面写了 "不要求 k 是最小正周期"，所以只要每个环长整除 k ，连 $\text{lcm} = k$ 都不需要
- 小凯的疑惑只能处理两个数。三个以上质因子要用同余最短路 (墨墨的等式、跳楼机)
- 最可惜的是：模型对了之后没写完全背包：
 - 子任务 1、2 ($n, k \leq 1000$)：对 k 的约数做完全背包
 - 子任务 3 ($n \leq 10^6$)：对质因子做完全背包，不同的 k 最多 50 个
 - 一共 $10+15+20 = 45$ 分，二十行代码
- 你最后写的 T4pian.cpp 是枚举全排列模拟， $n > 10$ 就跑不动，而且没写完

正解：先筛出 3.2×10^7 以内的质数，对每个不同的 k 分解质因数。

- $k = 1$ ：全部 NO ($f(x) = x$ 不允许)
- 只有 1 个质因子 p ：看 n 能否被 p 整除
- 2 个质因子 $p < q$ ： $n = ap + bq$ ，令 $b \equiv n \cdot q^{-1} \pmod{p}$ 取最小值，检查 $bq \leq n$
- 3 个及以上：最小质因子 $p_1 \leq \sqrt[3]{10^{15}} = 10^5$ 。对模 p_1 的每个余数跑最短路， $\text{dist}[r]$ = 能凑出来的、模 p_1 余 r 的最小值。 n 可行当且仅当 $\text{dist}[n \bmod p_1] \leq n$

验证：3 组大样例通过； $n, k \leq 60$ 的 3600 组和背包暴力比对，全部一致。代码见 正解代码/D_轮换密钥.cpp。

下次识别信号：

- $f^k = \text{id} \rightarrow$ 排列 + 环长整除 k
- "用几种固定的数凑出一个很大的 n " $\rightarrow$ 同余最短路，模最小的那个数
- 模型推出来以后，先按子任务写最暴力的实现，再想优化

五、关键争议点

1. "八成是我机子内核少，跑的慢" (录音 2:23)

不成立，两个理由：

- 你的程序是单线程，核多核少对它没影响
- 同一份代码，我在一台正常的 4 核服务器上跑：n=2000 用 9.2 秒（你虚拟机里是 7.7 秒，虚拟机反而更快），n=5000 超过 60 秒

真正的原因是第四节 A 的 `distance` 让复杂度变成了 $O(n^3)$ 。

规则：大样例跑超过时限的 2 倍，就当复杂度错了，不要怪机器。

2. "第一题我觉得没有任何问题，不要改" (录音 3:48)

不成立。你在 T+2:12 亲眼看到 4-1 卡死，自己也说了"掩耳盗铃"。T+3:35 以后你还有 25 分钟：

- 去 T2 试随机化，放弃
- 去 T4 写全排列，没写完

同样 25 分钟，如果回 T1 把 multiset 换成树状数组（约 15 行），是 +80 分。

规则：已经交了但知道会 TLE 的题，优先级高于任何没思路的新题。

3. "T4 我掰不动" (录音 3:16)

你掰不动的是满分，不是 45 分。你推出"能否拆成若干个质数之和"的那一刻，背包就已经可以写了。

规则：模型推出来以后，先问自己"最暴力的实现能拿几个子任务"，写完交了再想优化。

六、排名

归档时没勾选"排行榜"，这场没有名次数据，没法做同分段比较。

下次归档请把"排行榜"和"关注用户"都勾上。

七、止损复盘

本该止损的时刻	信号	应该做什么	实际
T+0:06	算出"要 $O(n)$ "，但 A 是第一题	第一题要求 $O(n)$ 且还没思路时，回去再读一遍数据范围	直接开干

本该止损的时刻	信号	应该做什么	实际
T+0:30	IDE 出问题	立刻切到终端 g++ 编译，别跟 IDE 较劲	跟 IDE 耗了 23 分钟
T+0:58	样例输出 20 > 上限 10	公式结构错了，停止调符号，重新推	继续调了 56 分钟
T+1:16	大样例第 1 组就错	同上，或者先写 $O(n^3)$ 暴力拿 20 分保底	继续调
T+1:33	开始改初始值碰运气	同上	继续调
T+2:12	知道 4-1 卡死	花 5 分钟想"哪一步慢"，查 distance 的复杂度	交了就走
T+3:04	T4 模型已经推对	写背包	去 T2 骗分

T1 的暴力 $O(n^3)$ 是 20 分，你最终也就拿了 20 分。如果 T+0:10 先写暴力，这 20 分 10 分钟就到手，还能拿来对拍。

八、环境问题（详见 `tools/vm/给ComputerUseAgent的虚拟机调试手册.md`）

问题	录音时间	损失
Code::Blocks 编译运行的不是屏幕上的代码	55:26 至 57:28	约 10 分钟 + 心态
控制台弹两个窗口 / 一闪就退	43:26 至 49:22	约 10 分钟
关 IDE 时连环 Yes/No 弹窗	1:03 至 1:20 多次	零碎几分钟
没有自动缩进	59:39	心态
终端命令没法复制，每次手敲 g++/diff	1:15、2:19 至 2:21、3:40	敲错多次
鼠标消失	3:39:52	1 分钟
虚拟机卡顿	2:21、3:14	零碎
时区错 12 小时	全程	不影响比赛，影响事后分析

加起来 IDE 和环境直接吃掉约 30 分钟，还持续打击心态（"一切都在我计划当中，但这代码怎么不按照我计划执行"）。

`tools/vm/run.sh` 可以一条命令编译并跑全部样例，报 OK/WA/TLE 和耗时。有了它，T+2:09 你会直接看到 "3-1 TLE / 4-1 TLE / 5-1 TLE"，而不是手敲命令、等它卡死、再怪机器。

九、经验教训（可以直接加进 Skill）

本场最大教训

读数据范围要读到最后一个字。 $\sum n \leq X$ 决定复杂度，漏读它一次，毁掉一整场。

题型规则

1. 区间点对统计（逆序对等）不能前缀相减： $P(r) - P(l-1)$ 里多了跨界点对。要么固定一端增量维护，要么二维 DP
2. "前面有几个比我大" → 树状数组。 `std::distance` 在 set/map/multiset 上是 $O(n)$ ；要在 set 上 $O(\log n)$ 求排名，得用 pb_ds 的 tree
3. 构造题：先判 YES/NO（暴力打表、看 checker），再"前 $i-1$ 个排好了，加第 i 个"
4. 树上"需要一个邻居是 X " → 三状态支配型 DP
5. "用几种固定的数凑很大的 n " → 同余最短路
6. $f^k = id$ → 排列、环长整除 k ；"不要求最小周期"意味着连 lcm 都不用管

调试规则

1. 输出超过理论上限或低于下限 → 公式结构错，停止调参数
2. 同一处正负号改了两次还不对 → 停下来，在纸上用 $n=3$ 手算每个变量
3. 变量含义写在代码第一行注释，草稿纸和代码用同一个名字
4. 大样例跑超过时限 2 倍 → 当成复杂度错，不怪机器

比赛策略规则

1. 开赛 15 分钟通读四题，每题列出"一眼能拿的子任务"，比如 C 的 $L=n$ 、D 的背包
2. 每题先交暴力，再想正解
3. 知道会 TLE 的已交代码，优先级高于没思路的新题
4. 每道题开写前说一句"我预计这题能拿 X 分，最多花 Y 分钟"，到点就换题

十、行动清单

补题顺序（从最亏的开始）：

1. A：自己写一遍树状数组求逆序对，再写 $O(n^2)$ 正解，用 `run.sh` 跑大样例
2. C：先做洛谷 P2458 保安站岗，再回来写 C
3. D：先做洛谷 P2371 墨墨的等式（同余最短路模板），再写 D
4. B：先自己证明 $a_i \leq i$ 的必要性，再试着写归纳构造

下次比赛执行：

1. 赛前（5 分钟内）：`init.sh` 建好 T1 到 T4；测一次 `run.sh`；不再预建四份对拍
2. T+0 到 0:15：通读四题，每题记下数据范围最后一行和"送分子任务"
3. 每题：暴力 → 交 → 正解 → `run.sh` → 交
4. T+3:00：停止开新题，回头检查每道已交代码有没有已知的 TLE/WA

赛后复盘要准备的材料：

1. 录音（继续保持，这次的录音极其有用）
2. 虚拟机桌面导出（继续保持，别清回收站，旧版本都在里面）

3. 归档时勾上排行榜

4. 修好虚拟机时区，时间就不用手动对齐了